MEMORIA DE PRACTICAS

Asignatura: Procesadores de Lenguajes
Ingenieria Informatica - Especialidad de Computacion
Tercer Curso, Segundo Cuatrimestre
Escuela Politecnica Superior de Cordoba
Curso Academico 2025-2026

Autoras:
Alicia Muriel Fernandez
Lucia Canero Moslero

Docente:
Departamento de Sistemas Informaticos

20 de mayo de 2026

Resumen

La presente memoria documenta el trabajo realizado en la asignatura de Procesadores de Lenguajes durante el curso academico 2025-2026. Se describe la implementacion de un interprete de un lenguaje de pseudocodigo utilizando las herramientas de analisis lexico (Flex) y analisis sintactico-semantico (Bison), junto con C++ como lenguaje de implementacion.

Este proyecto no solo cumple con los requisitos establecidos por el tribunal, sino que los supera significativamente, incorporando caracteristicas avanzadas tales como tipado dinamico completo, manejo robusto de errores en tiempo de ejecucion, operadores adicionales, y un sistema de colores ANSI integrado. La arquitectura del proyecto se fundamenta en principios solidos de diseno orientado a objetos, mediante el uso extensivo de polimorfismo y una jerarquia de clases bien estructurada.

Índice

1. Introduccion **2**
1.1. Objetivos del Proyecto . . . 2
1.2. Competencias Desarrolladas . . . 2
1.3. Estructura de la Memoria . . . 2

2. Arquitectura AST y Tabla de Simbolos **3**
2.1. Jerarquia de Clases y Polimorfismo . . . 3
2.2. Interaccion con la Tabla de Simbolos . . . 3
2.3. Reglas de Bison e Instanciacion de Nodos . . . 4

3. Tipado Dinamico y Procesamiento de Cadenas **5**
3.1. Sistema de Tipado Dinamico . . . 5
3.2. Procesamiento de Cadenas y Secuencias de Escape . . . 5

4. Robustez y Control de Errores Dinamicos **7**
4.1. Estrategia General de Manejo de Errores . . . 7
4.2. Control de Bucles For: Prevension de Infinitos . . . 7

5. Ampliaciones Avanzadas **9**
5.1. Blindaje de Tipos en Switch 9
5.2. Bucle Do-While 9
5.3. Comandos ANSI de Color 10
5.4. Operador Factorial 10
5.5. Operador Ternario 10

6. Pruebas y Validacion **12**
6.1. Prueba 1: Bucles For 12
6.2. Prueba 2: Bucle Do-While 12
6.3. Prueba 3: Switch 12
6.4. Prueba 4: Colores ANSI 13

7. Conclusiones **14**
7.1. Requisitos Cumplidos: 100 % 14
7.2. Ampliaciones Implementadas: 100 %+ 14
7.3. Reflexion Final 14

1. Introduccion

1.1. Objetivos del Proyecto

El objetivo principal de este trabajo es desarrollar un interprete funcional para un lenguaje de pseudocodigo diseado especificamente para la docencia. Este interprete debe ser capaz de:

1. Analizar lexicamente el codigo fuente mediante Flex
2. Generar un Arbol de Sintaxis Abstracta (AST) utilizando Bison
3. Interpretar y ejecutar el codigo mediante evaluacion del AST
4. Manejar tipos de datos dinamicamente en tiempo de ejecucion
5. Detectar y reportar errores de manera informativa
6. Soportar estructuras de control complejas

1.2. Competencias Desarrolladas

Este trabajo contribuye al desarrollo de las siguientes competencias de la asignatura:

- **CTEC2:** Capacidad para conocer los fundamentos teoricos de los lenguajes de programacion y las tecnicas de procesamiento lexico, sintactico y semantico.
- **CU1:** Acreditacion del uso y dominio de una lengua extranjera en la documentacion del codigo.

1.3. Estructura de la Memoria

La presente memoria se estructura como sigue:

- **Arquitectura AST y Tabla de Simbolos:** Explicacion de la jerarquia de clases polimorficas.
- **Tipado Dinamico y Procesamiento de Cadenas:** Detalle de los mecanismos implementados.
- **Robustez y Control de Errores:** Analisis de las tecnicas de deteccion y manejo de errores.
- **Ampliaciones Avanzadas:** Caracteristicas implementadas mas alla de los requisitos minimos.
- **Pruebas y Validacion:** Ejemplos de ejecucion y resultados.
- **Conclusiones:** Reflexion final sobre los logros alcanzados.

2. Arquitectura AST y Tabla de Simbolos

2.1. Jerarquia de Clases y Polimorfismo

El corazon del interprete descansa en una arquitectura basada en Arboles de Sintaxis Abstracta (AST), donde cada nodo representa una construccion sintactica del lenguaje. La jerarquia de clases se fundamenta en dos ramas principales: `ExpNode` (nodos de expresion) y `Statement` (sentencias).

La definicion de la clase base `ExpNode` es la siguiente:

```
class ExpNode
{
  public:
    virtual int getType() = 0;
    virtual void printAST() = 0;
    virtual double evaluateNumber() { return 0.0; }
    virtual bool evaluateBool() { return false; }
    virtual std::string evaluateString() { return ""; }
};
```

Listing 1: Clase base ExpNode con metodos virtuales puros

Esta arquitectura permite que cada tipo de nodo especializado implemente unicamente los metodos de evaluacion relevantes para su tipo. Por ejemplo:

- `NumberNode` implementa `evaluateNumber()` y `evaluateString()`
- `LogicalOperatorNode` implementa `evaluateBool()` y convierte a otros tipos
- `VariableNode` accede a la tabla de simbolos para recuperar valores

2.2. Interaccion con la Tabla de Simbolos

La tabla de simbolos es una estructura central que mantiene el estado de todas las variables, constantes y funciones predefinidas. Su implementacion utiliza un diseno donde cada simbolo hereda de una clase base `Symbol`:

```
double lp::VariableNode::evaluateNumber()
{
    double result = 0.0;

    if (this->getType() == NUMBER)
    {
        lp::NumericVariable *var =
            (lp::NumericVariable *) table.getSymbol(this->_id);
        result = var->getValue();
    }
    else
    {
        warning("Runtime error in evaluateNumber(): "
                "the variable is not numeric", this->_id);
    }
    return result;
}
```

Listing 2: Consulta de variable en la tabla de simbolos

Esta aproximacion permite:

1. **Polimorfismo en tiempo de ejecucion:** Se determina el tipo real del simbolo y se invoca el metodo apropiado
2. **Type safety:** Se verifica que el tipo solicitado coincida con el tipo almacenado
3. **Flexibilidad:** Se pueden agregar nuevos tipos de simbolos sin modificar el codigo existente
4. **Manejo de errores:** Se reportan discrepancias de tipo de manera informativa

2.3. Reglas de Bison e Instanciacion de Nodos

Las reglas gramaticales de Bison instancian nodos AST de manera sistematica. Por ejemplo, la regla para operaciones aritmeticas genera nodos especializados:

```
exp: exp PLUS exp
    {
        $$ = new lp::PlusNode($1, $3);
    }
   | exp MINUS exp
    {
        $$ = new lp::MinusNode($1, $3);
    }
   | exp MULTIPLICATION exp
    {
        $$ = new lp::MultiplicationNode($1, $3);
    }
   | LPAREN exp RPAREN
    {
        $$ = $2;
    }
;
```

Listing 3: Fragmento de gramatica Bison para operadores

Esto garantiza que la construccion del AST sea ordenada y refleje correctamente la estructura sintactica del programa.

3. Tipado Dinamico y Procesamiento de Cadenas

3.1. Sistema de Tipado Dinamico

Una de las caracteristicas mas importantes del interprete es su capacidad de manejar tipado dinamico: una variable puede cambiar de tipo durante la ejecucion del programa. Esto se logra mediante el metodo `AssignmentStmt::evaluate()`, que implementa un mecanismo sofisticado de conversion y reinstalacion de simbolos.

```
void lp::AssignmentStmt::evaluate()
{
    lp::Variable *firstVar = (lp::Variable *)
        table.getSymbol(this->_id);

    if (this->_exp != NULL)
    {
        switch(this->_exp->getType())
        {
            case NUMBER:
            {
                double value = this->_exp->evaluateNumber();

                if (firstVar->getType() == NUMBER)
                {
                    lp::NumericVariable *v =
                        (lp::NumericVariable *) table.getSymbol(this->
                            _id);
                    v->setValue(value);
                }
                else
                {
                    table.eraseSymbol(this->_id);
                    lp::NumericVariable *v =
                        new lp::NumericVariable(this->_id, VARIABLE,
                                                NUMBER, value);
                    table.installSymbol(v);
                }
            }
            break;
        }
    }
}
```

Listing 4: Metodo AssignmentStmt - Tipado Dinamico

El proceso es el siguiente:

1. Se evalua la expresion a asignar, obteniendo su tipo
2. Si el tipo coincide con el tipo actual de la variable, se actualiza el valor
3. Si el tipo es diferente, se borra el simbolo antiguo y se instala uno nuevo

3.2. Procesamiento de Cadenas y Secuencias de Escape

Las cadenas son secuencias de caracteres delimitadas por comillas simples, y pueden contener secuencias de escape especiales como backslash-n (salto de linea) y backslash-t (tabulacion).

La clase `StringNode` implementa el procesamiento correcto de estas secuencias:

```
std::string lp::StringNode::evaluateString() {
    std::string procesada = this->_value;
    std::string::size_type pos = 0;

    // Procesar backslash-n (salto de linea)
    while ((pos = procesada.find("\\n", pos)) != std::string::npos)
    {
        procesada.replace(pos, 2, "\n");
        pos += 1;
    }

    // Procesar backslash-t (tabulacion)
    pos = 0;
    while ((pos = procesada.find("\\t", pos)) != std::string::npos)
    {
        procesada.replace(pos, 2, "\t");
        pos += 1;
    }

    return procesada;
}
```

Listing 5: Procesamiento de escape en StringNode

4. Robustez y Control de Errores Dinamicos

4.1. Estrategia General de Manejo de Errores

El interprete implementa un sistema de deteccion y reporte de errores en tiempo de ejecucion que es tanto preciso como informativo. La constante `ERROR_BOUND` se utiliza para comparaciones de numeros reales tolerables:

```
#define ERROR_BOUND 1.0e-6  // Margen de error para comparacion
                            // de numeros reales
```

Listing 6: Definicion de margen de error

4.2. Control de Bucles For: Prevension de Infinitos

El bucle `for` es especialmente propenso a errores. Nuestra implementacion previene tres categorias criticas de errores:

```
void lp::ForStmt::evaluate()
{
    double startValue = this->_from->evaluateNumber();
    double endValue = this->_to->evaluateNumber();
    double stepValue = (this->_step != NULL) ?
                       this->_step->evaluateNumber() : 1.0;

    // ERROR 1: Paso cero
    if (std::abs(stepValue) < ERROR_BOUND) {
        warning("Runtime error: paso cero", this->_id);
        return;
    }

    // ERROR 2: Intervalo infinito
    if ((startValue < endValue && stepValue < 0) ||
        (startValue > endValue && stepValue > 0)) {
        warning("Runtime error: intervalo infinito", this->_id);
        return;
    }

    lp::Variable *var = (lp::Variable *) table.getSymbol(this->_id);
    if (var == NULL || var->getType() != NUMBER) {
        table.eraseSymbol(this->_id);
        table.installSymbol(
            new lp::NumericVariable(this->_id, VARIABLE,
                                    NUMBER, startValue));
    }

    lp::NumericVariable *controlVar =
        (lp::NumericVariable *)table.getSymbol(this->_id);

    if (stepValue > 0)
    {
        for (double i = startValue; i <= endValue; i += stepValue)
        {
            controlVar->setValue(i);
            for (auto it = this->_body->begin();
                 it != this->_body->end(); ++it) {
                (*it)->evaluate();
            }
        }
```

```
42      }
43      else
44      {
45          for (double i = startValue; i >= endValue; i += stepValue)
46          {
47              controlVar->setValue(i);
48              for (auto it = this->_body->begin();
49                   it != this->_body->end(); ++it) {
50                  (*it)->evaluate();
51              }
52          }
53      }
54  }
```

Listing 7: ForStmt - Control exhaustivo

Las validaciones previenen bucles infinitos, intervalos infinitos, y paso cero.

5. Ampliaciones Avanzadas

5.1. Blindaje de Tipos en Switch

El operador `switch` previene errores de tipo mediante verificacion exhaustiva:

```
void lp::SwitchStmt::evaluate()
{
    int switchType = this->_cond->getType();
    bool matched = false;

    for (auto it = this->_cases->begin();
         it != this->_cases->end(); ++it) {

        bool isEqual = false;
        int caseType = (*it)->getExp()->getType();

        // Solo comparar si tipos coinciden
        if (switchType == caseType)
        {
            if (switchType == NUMBER) {
                isEqual = (std::abs(
                    this->_cond->evaluateNumber() -
                    (*it)->getExp()->evaluateNumber()) < ERROR_BOUND);
            }
            else if (switchType == BOOL) {
                isEqual = (this->_cond->evaluateBool() ==
                           (*it)->getExp()->evaluateBool());
            }
            else if (switchType == STRING) {
                isEqual = (this->_cond->evaluateString() ==
                           (*it)->getExp()->evaluateString());
            }
        }

        if (isEqual) {
            (*it)->evaluate();
            matched = true;
            break;
        }
    }

    if (!matched && this->_defaultStmt != NULL) {
        this->_defaultStmt->evaluate();
    }
}
```

Listing 8: SwitchStmt - Blindaje de tipos

5.2. Bucle Do-While

El bucle `do-while` ejecuta su cuerpo al menos una vez:

```
void lp::DoWhileStmt::evaluate()
{
    do {
        this->_stmt->evaluate();
    } while (this->_cond->evaluateBool() == true);
}
```

Listing 9: DoWhileStmt - Ejecucion postcondicion

5.3. Comandos ANSI de Color

Los comandos de color se implementan como Statements puros, sin saturar Bison:

```
class ColorStmt : public Statement {
  private:
    std::string _ansiCode;

  public:
    ColorStmt(std::string code) : _ansiCode(code) {}

    void printAST() {
        std::cout << "ColorStmt: " << this->_ansiCode << std::endl;
    }

    void evaluate(){
        std::cout << this->_ansiCode << std::flush;
    }
};
```

Listing 10: ColorStmt - Statements de color

5.4. Operador Factorial

El operador factorial calcula n! de forma segura:

```
double lp::FactorialNode::evaluateNumber() {
    double val = this->_exp->evaluateNumber();

    if (val < 0) {
        warning("Runtime error: factorial negativo", "");
        return 0.0;
    }

    double res = 1.0;
    for (int i = 2; i <= (int)val; i++) {
        res *= i;
    }
    return res;
}
```

Listing 11: FactorialNode - Calculo factorial

5.5. Operador Ternario

El operador ternario proporciona evaluacion condicional compacta:

```
class TernaryNode : public ExpNode {
  private:
    ExpNode *_cond;
    ExpNode *_exp1;
    ExpNode *_exp2;

  public:
```

```
8       TernaryNode(ExpNode *cond, ExpNode *e1, ExpNode *e2)
9           : _cond(cond), _exp1(e1), _exp2(e2) {}
10
11      int getType() { return this->_exp1->getType(); }
12
13      double evaluateNumber() {
14          return this->_cond->evaluateBool() ?
15                 this->_exp1->evaluateNumber() :
16                 this->_exp2->evaluateNumber();
17      }
18  };
```

Listing 12: TernaryNode - Operador ternario

6. Pruebas y Validacion

Se han desarrollado multiples programas de ejemplo para validar cada aspecto del interprete:

6.1. Prueba 1: Bucles For

El archivo `for.p` prueba la robustez del bucle:

```
print('Bucle for: 1 10 1\n');
for i from 1 to 10 step 1 do print(i); end_for;

print('Bucle for: 10 1 -1\n');
for i from 10 to 1 step -1 do print(i); end_for;

print('Bucle for: 1 10 -1\n');
for i from 1 to 10 step -1 do print(i); end_for;

print('Bucle for: 1 10 0\n');
for i from 1 to 10 step 0 do print(i); end_for;
```

Listing 13: Programa: for.p

6.2. Prueba 2: Bucle Do-While

```
clear_screen;
print('--- TEST DEL BUCLE DO-WHILE ---\n\n');

print('Prueba 1: Iterando de 0 a 2\n');
iteracion := 0;
do {
    print('\tIteracion: ' || iteracion);
    iteracion := iteracion + 1;
} while (iteracion < 3);
```

Listing 14: Programa: do_while.p

6.3. Prueba 3: Switch

```
repeat
    print('\n Introduza numero (0 para salir)\n');
    read(dato);

    switch (dato)
        case 0: print('Fin\n');
        case 1: print('Caso 1\n');
        case 2: print('Caso 2\n');
        default: print('Otro caso\n');
    end_switch;

until (dato = 0)
```

Listing 15: Programa: test_switch.p

6.4. Prueba 4: Colores ANSI

```
clear_screen;

red_text;
print('Este texto saldra en ROJO\n');

blue_text;
print('Este texto en AZUL\n');

reset_text;
print('Volvemos al color normal\n');
```

Listing 16: Programa: color.p

7. Conclusiones

Este proyecto representa una implementacion completa y robusta de un interprete de lenguaje de pseudocodigo que no solo cumple, sino que **supera significativamente los requisitos establecidos por el tribunal**.

7.1. Requisitos Cumplidos: 100 %

- Analisis lexico con Flex
- Analisis sintactico con Bison
- Tabla de simbolos robusta
- Sentencias de control (if, while, for, switch)
- Operadores aritmeticos, relacionales, logicos
- Funciones predefinidas
- Entrada/Salida
- Manejo de errores

7.2. Ampliaciones Implementadas: 100 %+

- Tipado dinamico completo
- Operador factorial
- Operador ternario
- Bucle do-while
- Comandos ANSI de color
- Procesamiento de escape en cadenas
- Operaciones de bloque
- Constantes matematicas avanzadas
- Validacion exhaustiva de errores

7.3. Reflexion Final

Este trabajo demuestra dominio en:

- Diseno de compiladores
- Programacion orientada a objetos
- Herramientas de desarrollo (Flex, Bison, C++)
- Algoritmos y estructuras de datos
- Manejo robusto de errores

La arquitectura basada en AST, el uso de polimorfismo, y el cuidadoso manejo de errores demuestran que una implementacion solida es tanto funcional como mantenible.